

VISUSTWIN

Extensions Documentation

Digital Twin Extension Suite for NVIDIA Omniverse

Built on Kit SDK 110.0 / USD Composer

v0.1.0 | 2026-04-14

metaarchetech

Table of Contents

- 1. Overview - Architecture, Getting Started, Design System, Event Bus

ENVIRONMENT SIMULATION

- 2. Wind Tunnel - Position flow solver + curl noise visualization
- 3. Wind Analysis - Pedestrian comfort assessment + report generation
- 4. Light Compass - Solar path + illuminance rose chart

DATA CONNECTION

- 5. MQTT Bridge - Welltek IoT device data bridge
- 6. OSC Controller - OSC UDP receiver for scene control

SAFETY & MONITORING

- 7. Vision Detector - AI safety detection with YOLO

PRESENTATION

- 8. Exhibition Board - Second-screen display with viewport
- 9. Camera Travel - Cinematic camera flight control

DEV TOOLS

- 10. Dev REPL - Claude Code <> Kit file-based REPL

MANAGEMENT

- 11. Dashboard - Extension management hub + design system

1. Overview

Architecture & System Design

Digital Twin Extension Suite for NVIDIA Omniverse

Visustwin 是一組為 NVIDIA Omniverse Kit 開發的插件集合，專為建築數位孿生應用打造。

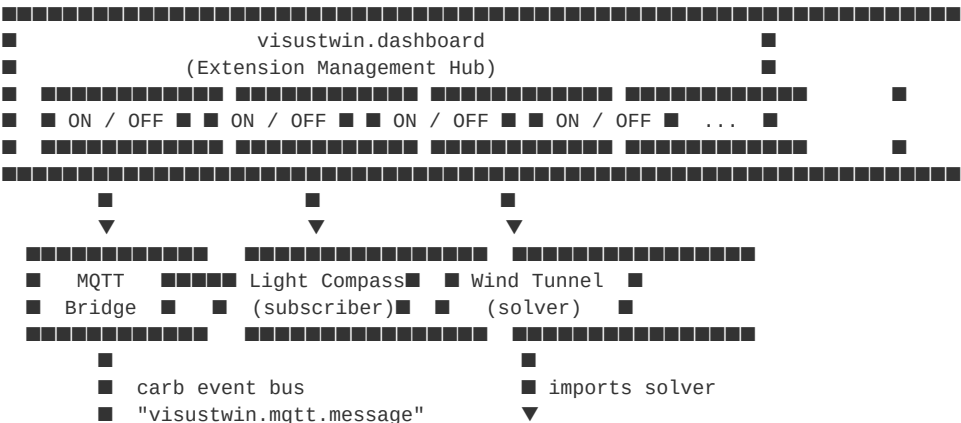
涵蓋環境模擬、IoT 數據串接、展示控制、AI 安全偵測等功能。

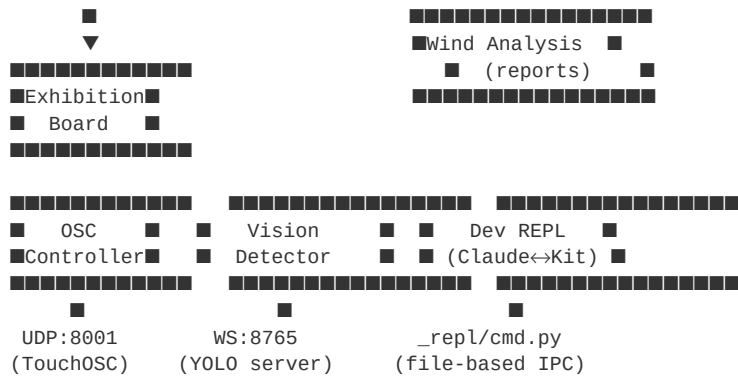
Built on NVIDIA Kit SDK 110.0 / Omniverse USD Composer
Developed by [metaarchetech](#)

Extension Overview

Category	Extension	Description
Environment Simulation	Wind Tunnel	建築群位勢流風場模擬，可調建築/風向/擾流
	Wind Analysis	風場分析與行人舒適度評估，自動產出圖表報告
	Light Compass	太陽軌跡弧 + 36 方向照度玫瑰圖，接 MQTT 感測器即時數據
Data Connection	MQTT Bridge	Welltek IoT 裝置數據橋接，WebSocket 接入
	OSC Controller	OSC UDP 接收器，外部觸控面板控制 USD 場景
Safety & Monitoring	Vision Detector	AI 安全偵測 overlay，串接 YOLO 即時影像分析伺服器
Presentation	Exhibition Board	第二螢幕展覽看板，獨立 OS 視窗 + Viewport
	Camera Travel	攝影機飛行控制面板，一鍵平滑飛往指定攝影機
Dev Tools	Dev REPL	Claude Code ↔ Kit 檔案式 Python REPL
Management	Dashboard	插件總覽與即時開關面板

Architecture





Getting Started

Prerequisites

- NVIDIA Omniverse Kit SDK 110.0+
- Python 3.10 (bundled with Kit)
- Windows 10/11

Installation

1. Clone this repository:
`git clone https://github.com/metaarchetech/visustwin-extensions.git`

2. Register the extension search path in your `.kit` file:

```
[settings.app.exts]  
folders.'++' = ["path/to/visustwin-extensions/exts"]
```

3. Enable extensions in the `.kit` manifest:

```
[dependencies]  
"visustwin.dashboard"           = {}  
"visustwin.mqtt.bridge"         = {}  
"visustwin.osc.controller"      = {}  
"visustwin.exhibition.board"    = {}  
"visustwin.warp.windtunnel"     = {}  
"visustwin.wind.analysis"       = {}  
"visustwin.light.compass"       = {}  
"visustwin.camera.travel"       = {}  
"visustwin.dev.repl"           = {}  
"visustwin.vision.detector"     = {}
```

Quick Start

啟動 Omniverse Composer 後，Dashboard 會自動開啟，列出所有可用的 Visustwin 插件。

透過 Dashboard 的 ON/OFF 按鈕即可啟用或停用各插件；VIEW 按鈕可切換各插件的 UI 面板。

Dependencies

Extension	Additional Dependency	Notes
<code>visustwin.warp.windtunnel</code>	<code>omni.warp.core</code>	Kit SDK >= 106 內建
<code>visustwin.wind.analysis</code>	<code>visustwin.warp.windtunnel</code>	匯入 solver 模組
<code>visustwin.exhibition.board</code>	<code>omni.kit.viewport.window</code> , <code>omni.kit.widget.viewport</code>	獨立 Viewport 需要

Extension	Additional Dependency	Notes
<code>visustwin.mqtt.bridge</code>	<code>websocket-client</code>	首次啟動透過 <code>omni.kit.pipapi</code> 自動安裝
<code>visustwin.vision.detector</code>	<code>websocket-client</code>	首次啟動透過 <code>omni.kit.pipapi</code> 自動安裝
<code>visustwin.osc.controller</code>	(none)	內建純 socket OSC parser

Shared Design System

所有插件共用統一的設計系統，定義於 `visustwin.dashboard.theme` 模組。

Color Palette

Token	Hex	Description
<code>BG_WIN</code>	<code>#F8F8F8</code>	視窗底色 (暖白)
<code>BG_HEADER</code>	<code>#1A1A1A</code>	標題列 (深灰)
<code>C_ACCENT</code>	<code>#1E7A4A</code>	品牌色 (綠色, 唯一彩色)
<code>C_PRIMARY</code>	<code>#1A1A1A</code>	主要文字
<code>C_SECONDARY</code>	<code>#666666</code>	次要文字
<code>C_ERROR</code>	<code>#CC3333</code>	錯誤狀態

Typography

Token	Size	Usage
<code>FONT_TITLE</code>	18px	主標題
<code>FONT_H2</code>	15px	段落標題
<code>FONT_BODY</code>	14px	內文
<code>FONT_LABEL</code>	13px	標籤
<code>FONT_CAPTION</code>	11px	註解

UI Helpers

```
from visustwin.dashboard.theme import (
    build_header, build_section_title, build_divider,
    btn_primary, btn_secondary, btn_danger, btn_toggle,
)
```

每個插件都會嘗試 import theme，失敗時使用內建 fallback 值，確保獨立運作能力。

Event Bus Communication

插件間透過 Carbonite (carb) event bus 進行非耦合通訊：

Event Type	Source	Subscribers	Payload
<code>visustwin.mqtt.message</code>	mqtt.bridge	light.compass, exhibition.board	<code>{topic, unit, payload, raw}</code>
<code>visustwin.osc.message</code>	osc.controller	(any)	<code>{address, args}</code>

Event Type	Source	Subscribers	Payload
<code>visustwin.vision.detection</code>	vision.detector	(any)	<code>{camera_id, count, raw}</code>
<code>visustwin.vision.alert</code>	vision.detector	(any)	<code>{alert_id, severity, category, camera_id, message}</code>

Subscribing Example

```
import carb.events
import omni.kit.app

EVENT_MQTT = carb.events.type_from_string("visustwin.mqtt.message")

sub = (
    omni.kit.app.get_app()
    .get_message_bus_event_stream()
    .create_subscription_to_pop_by_type(
        EVENT_MQTT, on_mqtt_event, name="my_subscriber"
    )
)

def on_mqtt_event(event):
    topic = event.payload.get("topic", "")
    raw = event.payload.get("raw", {})
    # ...
```

Dashboard Integration Protocol

所有插件遵循統一的 Dashboard 整合協議，使 Dashboard 能自動發現並控制各插件。

Required Elements

每個插件必須提供三個元素：

Element	Type	Description
<code>_INSTANCE</code>	module variable	模組層級實例參考，Dashboard 透過 <code>sys.modules</code> 查找
<code>_open_window()</code>	method	Dashboard 的 VIEW 按鈕呼叫此方法開啟 UI
<code>_window</code>	property	回傳具有 <code>.visible</code> 屬性的物件

Registration in Dashboard

```
# In visustwin.dashboard.extension:
_DESCRIPTIONS = { "your.ext.name": "████" }
_NAMES_EN      = { "your.ext.name": "DISPLAY NAME" }
_CLASS_MAP     = { "your.ext.name": "YourExtensionClass" }
```

並加入 `_CATEGORIES` 中對應的 category group。

File Structure

```
visustwin-extensions/
████ README.md           ← █████
████ exts/
  █████ visustwin.dashboard/
    █ █████ config/extension.toml
    █ █████ visustwin/dashboard/
      █ █████ extension.py      ← DashboardExtension
      █ █████ theme.py          ← Shared Design System
```

```

■■■ visustwin.camera.travel/
■■■ visustwin.mqtt.bridge/
■■■ visustwin.osc.controller/
■■■ visustwin.light.compass/
■■■ visustwin.warp.windtunnel/
■■■ visustwin.wind.analysis/
■■■ visustwin.exhibition.board/
■■■ visustwin.dev.repl/
■■■ visustwin.vision.detector/

```

Each extension follows:

```

visustwin.<name>/
■■■ config/
■   ■■■ extension.toml           # Manifest (title, deps, settings)
■■■ visustwin/<module>/
    ■■■ __init__.py
    ■■■ extension.py             # IExt class (on_startup / on_shutdown)
    ■■■ ui_window.py            # (optional) UI panel
    ■■■ <domain>.py             # (optional) Solver / client / animator

```

Version

v0.1.0

License

MIT

visustwin.warp.windtunnel

提供互動式風場模擬環境，使用位勢流解析解搭配 curl noise 擾流，

在 USD Stage 中即時渲染流線與建築物。可調整建築配置、風向、擾流強度等參數。

For Users

- ## For Developers

- ## UI Overview

[illegible]

Solver

Potential Flow Model

使用二維位勢流解析解計算風速場：

- 均勻流：基底風場（由風向角度決定）
- 建築繞流：每棟建築視為等效圓柱，產生偶極場
- 渦旋產生：建築後方產生 Kármán 渦旋效應
- Curl noise：疊加 curl noise 模擬大氣擾流

Color Modes

Mode	Description
Velocity	以風速大小著色
Pressure	以 Bernoulli 壓力著色
Vorticity	以渦度（curl）著色
Direction	以風向角度著色

Exported API

`solver.py` 提供以下函數供 Wind Analysis 等插件使用：

```
from visustwin.warp.windtunnel.solver import (
    compute_streamlines,
    sample_velocity_grid,
    sample_velocity_section,
    DEFAULT_BUILDINGS,
    colorize,
    COLOR_MODES,
)
```

USD Prim Structure

```
/World/WindTunnel/
■■■ Building_0      ← UsdGeom.Cube (transparent, no shadow)
■■■ Building_1
■■■ ...
■■■ Streamlines    ← UsdGeom.BasisCurves (colored by velocity)
```

Architecture

Key Classes

Class	File	Description
<code>WindTunnelExtension</code>	<code>extension.py</code>	主要 IExt 類別，UI + 建築管理
<code>WindTunnelAnimator</code>	<code>animator.py</code>	流線 USD 幾何建立與更新

Key Modules

Module	Description
<code>solver.py</code>	位勢流解析解、流線計算、grid sampling、色彩映射
<code>animator.py</code>	BasisCurves 建立、Points 粒子系統
<code>extension.py</code>	UI 建構、建築 CRUD、solver 參數管理

Key Functions (solver.py)

Function	Description
<code>compute_streamlines()</code>	計算流線（起點→終點軌跡）
<code>sample_velocity_grid()</code>	在 NxN 網格上取樣速度場
<code>sample_velocity_section()</code>	沿指定角度取樣垂直截面
<code>colorize()</code>	根據色彩模式將速度映射為 RGB

File Structure

```
visustwin.warp.windtunnel/
├── config/
│   └── extension.toml
├── visustwin/
│   └── warp/
│       └── windtunnel/
│           ├── __init__.py
│           ├── extension.py      ← WindTunnelExtension (UI + buildings)
│           ├── solver.py        ← Potential flow solver (exported API)
│           ├── animator.py      ← USD geometry creation
│           └── preview.py        ← Standalone preview script
```

Dependencies

- `omni.warp.core` (NVIDIA Warp GPU compute)
- `omni.usd`
- `omni.kit.uiapp`

visustwin.wind.analysis

匯入 Wind Tunnel 的 solver 模組，在指定高度上取樣風速場，計算行人舒適度指標並自動產出分析圖表報告。

For Users

- ## For Developers

- ## UI Overview

```

#####
■          WIND ANALYSIS          ■
■    Environmental Assessment    ■
#####
■ Sampling 100x100 grid...      ■
■                               ■
■ Analysis Parameters            ■
■ Wind Dir      [■●■■■■] 45°    ■
■ Turbulence    [●■■■■■] 0.15   ■
■ Sample Z      [■●■■■■] 150cm  ■
■ Ref Speed     [■●■■■■] 10m/s   ■
■ Grid Res      [■●■■■■] 100x    ■
■                               ■
■                               ■
■ Advanced Solver Params        ■
■ Vortex Str    [■●■■■■] 1.0     ■
■ Noise Scale   [■●■■■■] 1.0     ■
■ Vert Defl     [■■■●■■] 1.5     ■
■                               ■
■ Buildings (5)                  ■
■ Building_0 100x60x120         ■
■ [Sync from Wind Tunnel]      ■
■                               ■
■ [Generate Analysis Report]    ■
■ [View Last Report]           ■
#####

```

Report Output

每次分析產出以下檔案，存放於 extension 目錄下的 `output/` 資料夾：

File	Description
<code>speed_map.png</code>	風速分佈圖
<code>amplification_map.png</code>	風速放大倍率圖
<code>comfort_map.png</code>	行人舒適度分級圖
<code>pressure_map.png</code>	壓力分佈圖
<code>vector_field.png</code>	風向向量場圖
<code>summary.json</code>	統計數據摘要

報告資料夾命名格式：`wind_{direction}deg_{timestamp}/`

Parameters

Analysis Parameters

Parameter	Range	Default	Description
Wind Dir	0-360 °	45 °	風向角度
Turbulence	0-0.50	0.15	擾流強度
Sample Z	50-500 cm	150 cm	取樣高度（行人高度）
Ref Speed	1-30 m/s	10 m/s	參考風速
Grid Res	50-200	100	網格解析度（NxN）

Advanced Solver Parameters

Parameter	Range	Default	Description
Vortex Str	0-3.0	1.0	渦旋強度係數
Noise Scale	0.5-3.0	1.0	Curl noise 尺度
Vert Defl	0-3.0	1.5	垂直偏轉係數

Building Sync

點擊 "Sync from Wind Tunnel" 會從 USD Stage 的 `/World/WindTunnel/` 下讀取所有 `Building_*` Cube：

```
/World/WindTunnel/Building_0 → { cx, cy, wx, wy, hz, angle }
```

如果沒有同步建築，會使用 `solver.DEFAULT_BUILDINGS` 預設配置。

Architecture

Key Classes

Class	File	Description
<code>WindAnalysisExtension</code>	<code>extension.py</code>	主要 IExt 類別，UI + 分析流程
<code>ReportWindow</code>	<code>report_window.py</code>	圖表報告彈出視窗

Key Functions

Function	File	Description
<code>_generate_async()</code>	<code>extension.py</code>	非同步分析流程 (sync → sample → chart → report)
<code>generate_report()</code>	<code>engine.py</code>	產出 PNG 圖表 + JSON 統計
<code>sample_velocity_grid()</code>	<code>solver</code> (imported)	NxN 網格速度取樣
<code>sample_velocity_section()</code>	<code>solver</code> (imported)	垂直截面取樣

Analysis Pipeline

1. Sync buildings (from USD or defaults)
2. Import solver from `visustwin.warp.windtunnel`
3. Compute grid bounds from building positions
4. Sample NxN velocity grid at `z_height`
5. Sample 2 vertical sections (along-wind + cross-wind)
6. Generate 5 charts + `summary.json` via `engine.py`
7. Open `ReportWindow` with results

File Structure

```
visustwin.wind.analysis/
  ■■■ config/
  ■   ■■■ extension.toml
  ■■■ output/                                ← Generated reports (auto-created)
  ■   ■■■ wind_45deg_20260414_153000/
  ■       ■■■ speed_map.png
  ■       ■■■ amplification_map.png
  ■       ■■■ comfort_map.png
  ■       ■■■ pressure_map.png
  ■       ■■■ vector_field.png
  ■       ■■■ summary.json
  ■■■ visustwin/
  ■   ■■■ wind/
  ■       ■■■ analysis/
  ■           ■■■ __init__.py
  ■           ■■■ extension.py        ← WindAnalysisExtension
  ■           ■■■ engine.py          ← Chart generation + statistics
  ■           ■■■ report_window.py   ← Pop-out report viewer
```

Dependencies

- `visustwin.warp.windtunnel` (imports solver)
- `omni.usd`
- `omni.kit.uiapp`

4. Light Compass

visustwin.light.compass

方位照度視覺化－太陽軌跡弧+36方向照度玫瑰圖

在 USD Stage 中繪製太陽軌跡弧線與 36 方向照度玫瑰圖，

可接收 MQTT Bridge 廣播的即時感測器照度 (lux) 數據。

Features

For Users

- 太陽軌跡弧線視覺化（基於緯度/經度/時間的天文計算）
- 36 方向照度玫瑰圖（compass rose）
- 即時接收 MQTT 感測器照度數據
- 顯示當前太陽方位角與仰角
- 啟動/停止/重新建立按鈕
- 夜間偵測（仰角 < 0 時提示）

For Developers

- 天文太陽位置計算 (`solar.py`)
- USD BasisCurves + Points 幾何建立 (`animator.py`)
- 訂閱 `visustwin.mqtt.message` carb event (僅接收 `env` 類型的 `illuminance`)
- Frame update 驅動的動畫系統

UI Overview

[illegible]

MQTT Integration

Light Compass訂閱MQTT Bridge的visustwin.mqtt.message event：

```
EVENT_MQTT = carb.events.type_from_string("visustwin.mqtt.message")
```

僅處理 `topic == "env"` 的訊息，提取 `data.illuminance` 欄位更新照度數據。

USD Prim Structure

所有幾何建立於 `/World/LightCompass/` 下：

```

/World/LightCompass/
■■■ SunArc          ← ■■■■ (BasisCurves)
■■■ SunMarker       ← ■■■■■■■■

```

```

■■■■ CompassRose/      ← 36 ■■■■■
■   ■■■■ Dir_000
■   ■■■■ Dir_010
■   ■■■■ ...
■■■■ Labels/           ← ■■■■ (N/S/E/W)

```

關閉插件或重新建立時會自動清除 `/world/LightCompass` prim。

Architecture

Key Classes

Class	File	Description
<code>LightCompassExtension</code>	<code>extension.py</code>	主要 IExt 類別，管理 lifecycle 與 MQTT 訂閱
<code>LightCompassAnimator</code>	<code>animator.py</code>	幾何建立與 frame-based 動畫

Key Functions

Function	File	Description
<code>sun_position()</code>	<code>solar.py</code>	計算當前太陽方位角與仰角
<code>set_illuminance()</code>	<code>animator.py</code>	更新照度值，調整玫瑰圖高度
<code>setup()</code>	<code>animator.py</code>	在 Stage 上建立所有幾何
<code>start()/stop()</code>	<code>animator.py</code>	啟動/停止 frame update 訂閱

File Structure

```

visustwin.light.compass/
■■■■ config/
■   ■■■■ extension.toml
■■■■ visustwin/
    ■■■■ light/
        ■■■■ compass/
            ■■■■ __init__.py      ← LightCompassExtension
            ■■■■ extension.py    ← Compass rose + sun arc geometry
            ■■■■ animator.py     ← 
            ■■■■ solar.py        ← Solar position calculation

```

Dependencies

- `omni.usd`
- `omni.kit.uiapp`
- (subscribes to events from `visustwin.mqtt.bridge`)

5. MQTT Bridge

visustwin.mqtt.bridge

Welltek IoT 裝置數據橋接 — WebSocket 即時接入

連接 welltek-twin WebSocket server，接收 IoT 裝置即時數據（智慧插座、環境感測器、空調、HRV），並透過 Carbonite event bus 廣播至其他 Visustwin 插件。

Features

For Users

- 自動連接 Welltek WebSocket server（預設 `localhost:3001`）
- 即時顯示裝置數據摘要
- 支援四種裝置類型：智慧插座 (plug)、環境感測器 (env)、空調 (ac)、HRV 新風機
- 連線狀態指示
- 首次啟動自動安裝 `websocket-client` 套件

For Developers

- 透過 carb event bus 廣播 `visustwin.mqtt.message` 事件
- 其他插件可訂閱即時裝置數據（如 Light Compass 訂閱照度數據）
- Settings-based 設定，可在 `.kit` 設定檔中覆蓋

Device Types

Device	Key Fields	Summary Format
plug	power, current, voltage	120W 0.5A 220V
env	temperature, humidity, co2, illuminance	25°C 60% CO2 450ppm
ac	mode, set_temp, fan_speed	AC cool 25°C fan=3
hrv	fan_speed	HRV fan=2

Settings

Path	Type	Default	Description
<code>/visustwin/mqtt/broker_host</code>	string	<code>"localhost"</code>	WebSocket server 主機
<code>/visustwin/mqtt/broker_port</code>	int	<code>3001</code>	WebSocket server 埠
<code>/visustwin/mqtt/enabled</code>	bool	<code>true</code>	啟用自動連線

Runtime Status (read-only)

Path	Type	Description
<code>/visustwin/mqtt/connected</code>	bool	目前連線狀態

Path	Type	Description
<code>/visustwin/mqtt/last_topic</code>	string	最後接收的裝置名稱
<code>/visustwin/mqtt/last_payload</code>	string	最後接收的數據摘要

Event Bus

Published Event: `visustwin.mqtt.message`

每次收到裝置更新時推送：

```
{
  "topic": "env",          # ■■■■
  "unit": 1,              # ■■■■
  "payload": "25°C 60% CO2 450ppm", # ■■■■
  "raw": { ... }          # ■■■■■■
}
```

Subscribing

```
EVENT_MQTT = carb.events.type_from_string("visustwin.mqtt.message")
sub = bus.create_subscription_to_pop_by_type(EVENT_MQTT, handler, name="my_sub")
```

Architecture

Key Classes

Class	File	Description
<code>MqttBridgeExtension</code>	<code>extension.py</code>	主要 IExt 類別，管理連線與事件廣播
<code>WelltekWsClient</code>	<code>ws_client.py</code>	WebSocket 客戶端，背景 thread 運行
<code>WelltekUiWindow</code>	<code>ui_window.py</code>	狀態面板 UI

Data Flow

```
welltek-twin server (:3001)
  ■ WebSocket JSON
  ▼
WelltekWsClient (background thread)
  ■ callback
  ▼
MqttBridgeExtension._on_ws_message()
  ■■■ _summarise() → settings update
  ■■■ UI update (on_device_update)
  ■■■ carb event bus push → subscribers
```

File Structure

```
visustwin.mqtt.bridge/
■■■ config/
■ ■■■ extension.toml
■■■ visustwin/
    ■■■ mqtt/
        ■■■ bridge/
            ■■■ __init__.py
            ■■■ extension.py    ← MqttBridgeExtension
            ■■■ ws_client.py    ← WelltekWsClient (WebSocket)
            ■■■ ui_window.py    ← WelltekUiWindow
            ■■■ fake_data.py    ← Mock data for testing
```

Dependencies

- `omni.kit.uiapp`
- `omni.kit.pipapi` (auto-install `websocket-client`)

6. OSC Controller

visustwin.osc.controller

OSC UDP 接收器 — 外部觸控面板控制 USD 場景

監聽 UDP OSC 訊息（來自 iPad TouchOSC、手機 App 或其他 OSC 來源），
將指令映射到 Omniverse USD 場景操作：攝影機飛行、房間切換、燈光控制、時間軸播放。

Features

For Users

- 接收 OSC 1.0 格式 UDP 訊息（預設 port 8001）
- 攝影機即時切換與飛行控制
- 房間可見性切換（展示不同空間）
- 日夜燈光連續混合（0-100%）
- 動畫時間軸控制（播放/停止）
- 120 年生命週期時間軸
- 預設 BIM / Dataflow 攝影機巡迴路徑
- 即時顯示最後接收的 OSC 位址與參數

For Developers

- 純 socket + struct 實作的 OSC parser，無外部相依
- 背景 thread 監聽 UDP，透過 asyncio 排程到 Kit 主 thread
- 透過 carb event bus 廣播 `visustwin.osc.message`
- 可擴充的 dispatch 機制

Supported OSC Addresses

Address	Args	Action
<code>/test</code>	(none)	飛往測試攝影機（連線測試）
<code>/camera/<name></code>	(none)	飛往 <code>/World/Cameras/<name></code>
<code>/scene/room/<name></code>	(none)	顯示指定 Room，隱藏其他（ <code>/World/Rooms/</code> 下）
<code>/lighting/day</code>	(none)	日間燈光預設（blend = 0）
<code>/lighting/night</code>	(none)	夜間燈光預設（blend = 100）
<code>/lighting/blend</code>	float 0-100	日夜連續混合（0=日, 100=夜）
<code>/anim/play</code>	(none)	播放時間軸動畫
<code>/anim/stop</code>	(none)	停止時間軸動畫
<code>/sequence/lifecycle</code>	float 0-100	120 年生命週期位置（0-100% → 0-120 年）
<code>/sequence/bim</code>	(none)	觸發 BIM 巡迴攝影機序列

Address	Args	Action
<code>/sequence/dataflow</code>	(none)	觸發 Dataflow 視覺化攝影機序列

未辨識的位址會透過 carb event bus 廣播，供其他插件自行處理。

Settings

Path	Type	Default	Description
<code>/visustwin/osc/port</code>	int	8001	UDP 監聽埠（UE 預設用 8000）
<code>/visustwin/osc/enabled</code>	bool	true	啟用自動監聽
<code>/visustwin/osc/test_camera_path</code>	string	<code>/World/Cameras/Camera_01</code>	<code>/test</code> 指令的目標攝影機

Runtime Status (read-only)

Path	Type	Description
<code>/visustwin/osc/connected</code>	bool	Server 是否正在監聽
<code>/visustwin/osc/last_address</code>	string	最後接收的 OSC 位址
<code>/visustwin/osc/last_args</code>	string	最後接收的 OSC 參數

USD Stage Expectations

Room Switching (`/scene/room/<name>`)

期望 Stage 結構：

```

/World/Rooms/
  LivingRoom ← "LivingRoom"
  Bedroom
  Kitchen
  ...

```

Lighting Blend (`/lighting/blend`)

期望 Stage 結構：

```

/World/Lighting/
  DayLights ← blend=0
  NightLights ← blend=100

```

混合值介於 0-100 之間時，兩組燈光同時可見，透過 `displayOpacity` 控制透明度。

Camera Sequences (`/sequence/bim`, `/sequence/dataflow`)

期望攝影機命名：

```

/World/Cameras/Seq_bim_01
/World/Cameras/Seq_bim_02
/World/Cameras/Seq_dataflow_01
...

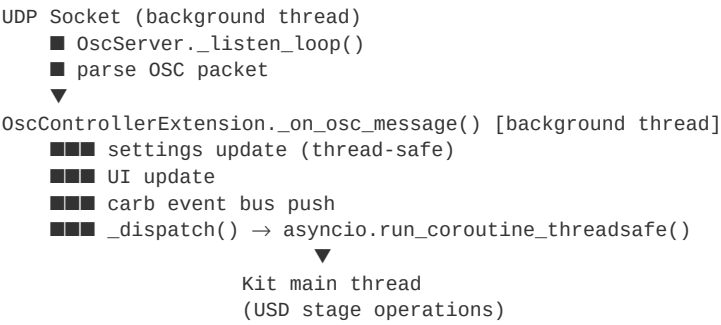
```

Architecture

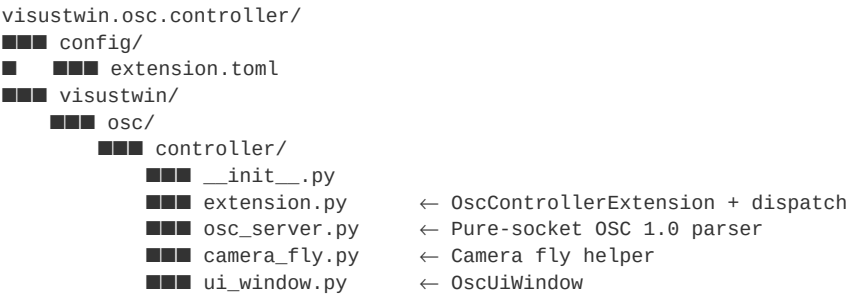
Key Classes

Class	File	Description
<code>OscControllerExtension</code>	<code>extension.py</code>	主要 IExt 類別, dispatch OSC 訊息
<code>OscServer</code>	<code>osc_server.py</code>	UDP OSC parser + receiver (pure socket)
<code>OscUiWindow</code>	<code>ui_window.py</code>	狀態面板 UI

Threading Model



File Structure



Dependencies

- `omni.kit.uiapp`
- (no external packages — OSC parser is built-in)

7. Vision Detector

visustwin.vision.detector

AI 安全偵測 Overlay — 串接 YOLO 即時影像分析伺服器

連接 visustwin-vision YOLO 伺服器，接收即時影像偵測結果（物件偵測、安全告警），
在 Omniverse Viewport 中顯示偵測 overlay 並透過 carb event bus 廣播。

Features

For Users

- 即時 AI 物件偵測結果顯示
- 安全告警通知（severity / category 分級）
- 多攝影機支援
- 連線狀態即時指示
- 首次啟動自動安裝 `websocket-client` 套件

For Developers

- WebSocket 客戶端連接 YOLO server（預設 `localhost:8765`）
- 透過 carb event bus 廣播偵測事件與告警事件
- Settings-based 設定，可在 `.kit` 設定檔中覆蓋
- 支援 `hello` / `detection_frame` / `alert` / `heartbeat` 訊息類型

Settings

Path	Type	Default	Description
<code>/visustwin/vision/server_host</code>	string	<code>"localhost"</code>	YOLO server 主機
<code>/visustwin/vision/server_port</code>	int	<code>8765</code>	YOLO server WebSocket 埠
<code>/visustwin/vision/enabled</code>	bool	<code>true</code>	啟用自動連線

Runtime Status (read-only)

Path	Type	Description
<code>/visustwin/vision/connected</code>	bool	目前連線狀態
<code>/visustwin/vision/last_camera</code>	string	最後偵測的攝影機 ID
<code>/visustwin/vision/last_detections</code>	string	最後偵測結果摘要

Message Types

`hello` (server → client)

連線建立後伺服器傳送，包含可用的攝影機與分析器清單：

```
{
  "type": "hello",
  "cameras": ["cam_01", "cam_02"],
  "analyzers": ["safety", "fire", "ppe"]
}
```

detection_frame (server → client)

每一幀的偵測結果：

```
{
  "type": "detection_frame",
  "camera_id": "cam_01",
  "detections": [
    { "class": "person", "confidence": 0.95, "bbox": [x, y, w, h] },
    { "class": "hardhat", "confidence": 0.88, "bbox": [x, y, w, h] }
  ]
}
```

alert (server → client)

安全告警觸發時：

```
{
  "type": "alert",
  "alert_id": "alert_001",
  "severity": "high",
  "category": "no_hardhat",
  "camera_id": "cam_01",
  "message": "Person detected without hardhat in zone A"
}
```

heartbeat (server → client)

定期心跳確認連線活躍。

Event Bus

Published Events

Event Type	Trigger	Payload
visustwin.vision.detection	每幀偵測結果	{camera_id, count, raw}
visustwin.vision.alert	安全告警	{alert_id, severity, category, camera_id, message}

Subscribing Example

```
EVENT_DET = carb.events.type_from_string("visustwin.vision.detection")
sub = bus.create_subscription_to_pop_by_type(EVENT_DET, handler, name="my_sub")

def handler(event):
    camera_id = event.payload.get("camera_id", "")
    count = event.payload.get("count", 0)
    raw = event.payload.get("raw", {})
```

Architecture

Key Classes

Class	File	Description
VisionDetectorExtension	extension.py	主要 IExt 類別，管理連線與事件廣播
VisionWsClient	ws_client.py	WebSocket 客戶端（背景 thread）
VisionUiWindow	ui_window.py	偵測結果 UI 面板

Data Flow

```

visustwin-vision YOLO server (:8765)
  ■ WebSocket JSON
  ▼
VisionWsClient (background thread)
  ■ callback
  ▼
VisionDetectorExtension._on_ws_message()
  ■■■ hello      → UI: camera/analyzer list
  ■■■ detection  → settings + UI + carb event bus
  ■■■ alert      → UI + carb event bus
  ■■■ heartbeat  → UI: connection alive

```

File Structure

```

visustwin.vision.detector/
  ■■■ config/
  ■ ■■■ extension.toml
  ■■■ visustwin/
    ■■■ vision/
      ■■■ detector/
        ■■■ __init__.py
        ■■■ extension.py ← VisionDetectorExtension
        ■■■ ws_client.py ← VisionWsClient (WebSocket)
        ■■■ ui_window.py ← VisionUiWindow

```

Dependencies

- [omni.kit.uiapp](#)
- [omni.kit.pipapi](#) (auto-install [websocket-client](#))

Related

- [visustwin-vision](#) — YOLO detection backend server

8. Exhibition Board

visustwin.exhibition.board

第二螢幕展覽看板 — 嵌入式 Viewport + Welltek 即時數據

提供獨立的 OS 視窗，內嵌 Omniverse Viewport 與即時裝置數據，適合展場使用的第二螢幕顯示方案。

Features

For Users

- 獨立 OS 視窗（脫離 Omniverse 主視窗，出現在工作列）
- 內嵌即時 3D Viewport
- 顯示 Welltek IoT 裝置即時數據
- 可透過 Dashboard 的 VIEW 按鈕開啟/關閉
- 自動前景顯示（Windows API bring-to-front）

For Developers

- 使用 `omni.kit.viewport.window` 建立獨立 Viewport
- `move_to_new_os_window()` 將 Kit 視窗提升為獨立 OS 視窗
- Windows `user32.dll` ctypes 呼叫控制視窗前景
- 非同步啟動流程（等待 render 完成後再移動視窗）

Window Lifecycle

```
_open_window()
  ■
  ▼
_async_show() ← asyncio.ensure_future
  ■
  ■■■ window.visible = True
  ■■■ wait 8 frames (let Kit render)
  ■■■ window.move_to_new_os_window()
  ■■■ wait 5 frames
  ■■■ _bring_to_front() → user32.dll
```

Bring to Front (Windows)

使用 ctypes 呼叫 Windows API：

```
import ctypes
user32 = ctypes.windll.user32
hwnd = user32.FindWindowW(None, "Visustwin ■■■■")
user32.ShowWindow(hwnd, 9)          # SW_RESTORE
user32.SetForegroundWindow(hwnd)
user32.BringWindowToTop(hwnd)
```

Architecture

Key Classes

Class	File	Description
<code>ExhibitionBoardExtension</code>	<code>extension.py</code>	主要 IExt 類別，管理視窗 lifecycle

Class	File	Description
ExhibitionBoardWindow	ui_window.py	自訂視窗類別，含 Viewport + 數據面板

File Structure

```

visustwin.exhibition.board/
├── config/
├── extension.toml
├── visustwin/
│   └── exhibition/
│       └── board/
│           ├── __init__.py
│           ├── extension.py    ← ExhibitionBoardExtension
│           └── ui_window.py    ← ExhibitionBoardWindow (Viewport + data)

```

Dependencies

- omni.kit.uiapp
- omni.kit.viewport.window
- omni.kit.widget.viewport
- omni.kit.viewport.utility
- omni.usd

9. Camera Travel

visustwin.camera.travel

攝影機飛行控制面板 — 點擊即滑順飛往指定攝影機

自動掃描場景中所有 `UsdGeom.Camera`，提供下拉選單與一鍵飛行功能。

使用 Hermite 平滑插值，實現電影級的攝影機過場動畫。

Features

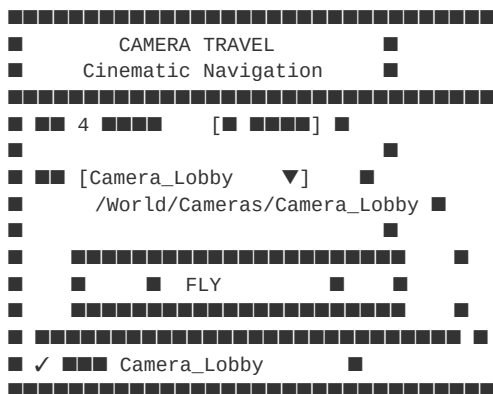
For Users

- 自動掃描 USD Stage 中所有 Camera prim
- 下拉選單即時切換目標攝影機
- 一鍵 FLY — 平滑飛行過場 (~1.5 秒)
- 飛行過程中按鈕狀態即時回饋 (飛行中 / 已抵達)
- 重新掃描按鈕，隨時更新攝影機列表
- Stage 開啟時自動建立預設攝影機 (Lobby / Living / Kitchen / Master)

For Developers

- Hermite 平滑插值 ($t * t * (3 - 2t)$)，90 frames @ 60fps
- Quaternion Slerp 旋轉插值
- 非同步 fly 動畫，可取消正在進行的飛行
- 獨立 `_fly_to()` function 可供其他插件呼叫
- Stage event 訂閱，自動偵測場景更換

UI Overview



Camera Auto-Discovery

Scanning

遍歷整個 USD Stage，收集所有 `UsdGeom.Camera` prim：

```
for prim in stage.Traverse():
    if prim.IsA(UsdGeom.Camera):
        results.append((prim.GetName(), str(prim.GetPath())))
```

Default Camera Creation

如果場景中的攝影機少於 4 個，自動在 `/World/Cameras/` 下建立預設攝影機：

Camera	Position	Rotation
Camera_Lobby	(0, 600, 180)	(75, 0, 180)
Camera_Living	(600, 0, 180)	(80, 0, 90)
Camera_Kitchen	(-600, 0, 180)	(80, 0, -90)
Camera_Master	(0, 0, 500)	(35, 0, 0)

Fly Animation

- Duration: 90 frames (~1.5s @ 60fps)
- Easing: Hermite smoothstep $t*t*(3-2t)$
- Position: Linear interpolation with smoothstep alpha
- Rotation: Quaternion Slerp (shortest path)
- Cancellation: 新的飛行會自動取消正在進行的飛行

Public API

其他插件可直接呼叫飛行功能：

```
from visustwin.camera.travel.extension import _fly_to
_fly_to("/World/Cameras/Camera_Lobby", duration_frames=90, on_done=callback)
```

Architecture

Key Classes

Class	File	Description
CameraTravelExtension	extension.py	主要 IExt 類別

Key Functions

Function	Description
<code>_fly_async()</code>	非同步攝影機飛行動畫 (coroutine)
<code>_fly_to()</code>	啟動飛行的公開介面
<code>_scan_cameras()</code>	掃描 Stage 中所有 UsdGeom.Camera
<code>_ensure_default_cameras()</code>	自動建立預設攝影機
<code>_smooth()</code>	Hermite 平滑插值函數

File Structure

```
visustwin.camera.travel/
■■■ config/
■   ■■■ extension.toml
■■■ visustwin/
    ■■■ camera/
        ■■■ travel/
            ■■■ __init__.py
            ■■■ extension.py    ← CameraTravelExtension + fly logic
```

Dependencies

- `omni.usd`
- `omni.kit.uiapp`

10. Dev REPL

visustwin.dev.repl

Claude Code ↔ Kit 檔案式 Python REPL

監視 `_repl/cmd.py` 的檔案修改，在 Kit Python 環境內即時執行，結果寫回 `_repl/result.txt`。用於 Claude Code 與 Omniverse Kit 之間的 IPC 通訊。

Features

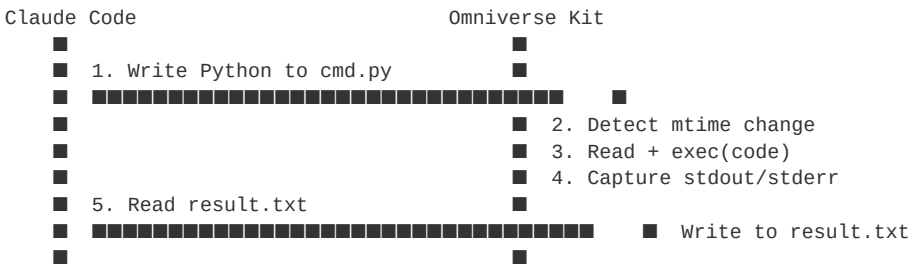
For Users

- 檔案式 REPL — 修改 `cmd.py` 即自動執行
- 在 Kit Python 環境中執行（完整 `omni.*`、`pxr`、`carb` 存取）
- 執行結果即時顯示（狀態、耗時、輸出）
- 手動觸發按鈕（不修改檔案也可重新執行）
- 首次啟動自動建立範例 `cmd.py`

For Developers

- 檔案式 IPC 機制，適合 AI Agent ↔ Kit 通訊
- 每 30 frames 輪詢 mtime 變化（~0.5 秒間隔）
- `exec()` 執行，可存取完整 Kit Python context
- stdout / stderr 完整捕捉
- 可自訂 REPL 目錄（透過 settings）

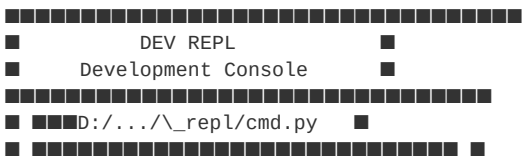
Workflow



IPC Files

File	Writer	Reader	Description
<code>_repl/cmd.py</code>	Claude Code	Kit	要執行的 Python 程式碼
<code>_repl/result.txt</code>	Kit	Claude Code	執行結果（狀態 + 輸出）

UI Overview



[illegible]

Settings

Path	Type	Default	Description
<code>/visustwin/repl/dir</code>	string	"" (auto-detect)	REPL 工作目錄，留空 = 自動偵測

Auto-Detection

如果 `dir` 設定為空，自動往 extension 目錄上層 6 層尋找 `_repl/` 資料夾：

```
pathlib.Path(__file__).resolve().parents[6] / "_repl"
```

Result Format

[illegible]

或在錯誤時：

```
[ERROR] 14:32:05 (0.5 ms)
Traceback (most recent call last):
  File "D:/.../cmd.py", line 3, in <module>
    ...
NameError: name 'foo' is not defined
```

Architecture

Key Classes

Class	File	Description
DevReplExtension	extension.py	主要 IExt 類別，輪詢 + 執行 + UI

Key Functions

Function	Description
<code>_on_update()</code>	Frame-based 輪詢（每 30 frames 檢查 mtime）
<code>_execute()</code>	讀取 <code>cmd.py</code> → <code>exec()</code> → 捕捉輸出 → 寫入 <code>result.txt</code>
<code>_write_result()</code>	將執行結果寫入 <code>result.txt</code>
<code>_trigger_now()</code>	手動觸發（touch cmd.py）

Execution Environment

```
exec(compile(code, str(CMD_FILE), "exec"), {"__name__": "__repl__"})
```

- 完整 Kit Python context (`omni.usd`, `pxr`, `carb` 等)

- `__name__` 設為 "`__repl__`" 以區別一般模組
- `stdout/stderr` 重導向至 `StringIO` 緩衝

File Structure

```
visustwin.dev.repl/
├── config/
│   └── extension.toml
├── visustwin/
│   └── dev/
│       └── repl/
│           ├── __init__.py
│           └── extension.py      ← DevReplExtension (poll + exec + UI)
```

Dependencies

- `omni.kit.uiapp` (optional, for menu registration)
- (no external packages)

11. Dashboard

visustwin.dashboard

插件總覽與即時開關面板

Dashboard 是 Visustwin 插件套件的管理中心，啟動後自動列出所有已安裝的 Visustwin 插件，提供即時 ON/OFF 切換與 UI 面板開關。

Features

For Users

- 自動掃描並列出所有 **visustwin.*** 插件
- 按類別分組顯示 (Environment Simulation / Data Connection / Safety & Monitoring / Presentation / Dev Tools)
- ON/OFF 按鈕即時啟用或停用插件
- VIEW 按鈕切換各插件的 UI 面板
- 即時狀態指示燈 (綠色 = 啟用)
- 顯示版本號與中文描述
- 底部統計：已載入 / 已啟用的插件數量

For Developers

- 提供共用設計系統 ([theme.py](#))，所有插件統一視覺風格
- 定義 Dashboard Integration Protocol，標準化插件發現與控制機制
- 訂閱 extension enable/disable 事件，UI 自動重建
- 啟動時可自動開啟指定 USD Stage (透過 `/visustwin/startup/stage_path` 設定)

UI Overview

[illegible]

Settings

Path	Type	Default	Description
<code>/visustwin/startup/stage_path</code>	string	""	啟動時自動開啟的 USD Stage 路徑

Design System (`theme.py`)

Dashboard 提供共用的設計系統模組，其他插件透過以下方式引入：

```
from visustwin.dashboard.theme import (
    BG_WIN, BG_HEADER, C_ACCENT, C_PRIMARY,
    FONT_TITLE, FONT_BODY, FONT_LABEL,
    SP_S, SP_M, MARGIN, RADIUS,
    build_header, btn_primary, btn_toggle,
)
```

所有顏色、字型、間距、按鈕樣式在此集中管理。

各插件會在 import 失敗時使用 inline fallback 值。

Architecture

Key Classes

Class	File	Description
<code>DashboardExtension</code>	<code>extension.py</code>	主要 IExt 類別，管理 UI 與插件狀態

Key Functions

Function	Description
<code>_build_ui()</code>	建構完整 UI，包含 header、category sections、extension cards
<code>_build_card()</code>	渲染單個插件卡片（名稱、版本、描述、VIEW、ON/OFF）
<code>_find_ext_instance()</code>	透過 <code>sys.modules</code> + <code>_INSTANCE</code> 查找執行中的插件實例
<code>_schedule_rebuild()</code>	插件狀態變更時排程非同步 UI 重建

Extension Discovery

Dashboard 透過以下三個 dict 管理插件：

```
_DESCRIPTIONS = { "ext.name": "■■■■" }
_NAMES_EN      = { "ext.name": "ENGLISH NAME" }
_CLASS_MAP     = { "ext.name": "ClassName" }
```

查找插件實例的流程：

1. 從 `_CLASS_MAP` 取得 class name
2. 在 `sys.modules` 中搜尋 `ext.name.extension` 或 `ext.name`
3. 檢查模組的 `_INSTANCE` 變數
4. 驗證 instance 的 class name 是否匹配

File Structure

```
visustwin.dashboard/
■■■ config/
■ ■■■ extension.toml
■■■ visustwin/
    ■■■ dashboard/
```

```
■■■ __init__.py
■■■ extension.py
■■■ theme.py
```

```
← DashboardExtension + _DESCRIPTIONS / _CLASS_MAP
← Shared Design System (colors, fonts, helpers)
```

Dependencies

- `omni.kit.uiapp`